Who needs language servers?

An experiment in generalizing language servers

Curtis Chin Jen Sem

Scan for these slides

Introduction

- I did my studies at UU
- I work at Scrive
  - A legaltech company specializing in digital agreements
- I like programming languages
- I help maintain Haskell Language Server
- I write infrequently at https://crtschin.nl/

Motivation

- Have some tricky memory or performance issues in a Haskell
- Have to look at GHC Core files
- Slowly start losing sanity points

[1] There are >20 flags for different phases, and another >15 flags for rendering

Motivation

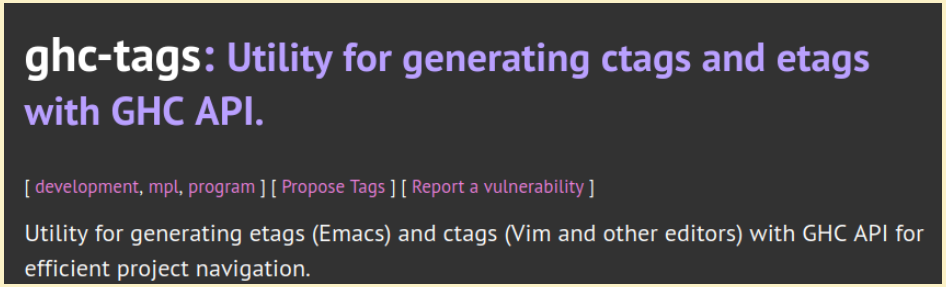
- Have some tricky memory or performance issues in a Haskell
- Have to look at GHC Core files
- Slowly start losing sanity points

- What if I can build tooling around GHC Core?
- Very easy semantics, but no[1] real syntax
- Impractical surface input to a language server

[1] There are >20 flags for different phases, and another >15 flags for rendering

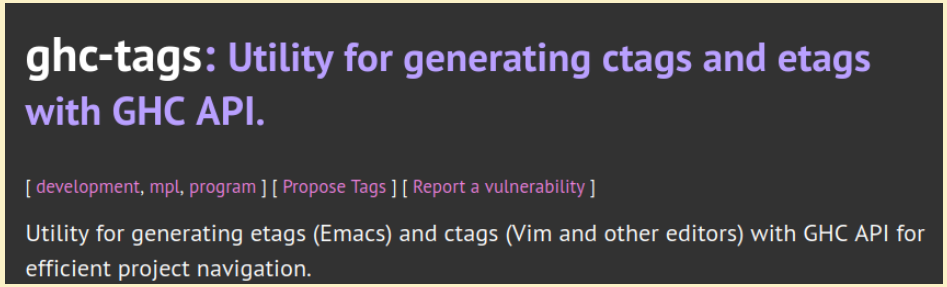
A question

- Who uses ghc-tags?



A question

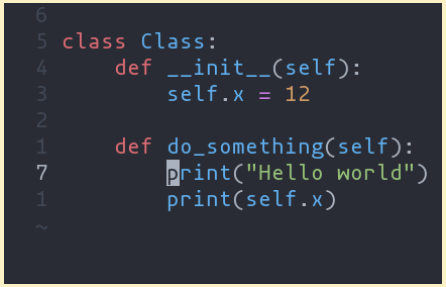
- Who uses ghc-tags?



- A GHC plugin for emitting `tags` files
- Useful for emacs and vim for goto-like functionality
- `tags` are language agnostic
- “Just” generate the tags file from Haskell

Second question

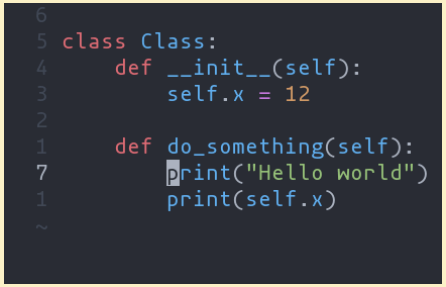
- Who's familiar with textobjects?[1]



[1] I've recently started using Helix, coming from VSCode, which inherits these from Vim

Second question

- Who's familiar with textobjects?[1]



- Structured text navigation in editors
- Uses tree-sitter queries
- Also language-agnostic

[1] I've recently started using Helix, coming from VSCode, which inherits these from Vim

So...

- Is it possible to write generic language servers?

So...

- Is it possible to write generic language servers?

- For “most” useful editor functionality you don’t need really much
- “Just” need to capture some semantics from parse-trees
- How about Tree-sitter + LSP?

Crash slide: Tree-sitter

- A parser generator DSL for incremental parsers
- Grammar in javascript, generates parsers in C
- Out comes a concrete syntax tree

```
library
    build-depends: base ≥ 4.9
```

```
(library
  type: (section_type)
  properties: (property_or_conditional_block
    (field
      name: (field_name)
      value: (field_value
        (identifier)
        (constraint_op)
        (version)))))
```

Crash slide: Tree-sitter

- Used to support editor-functionality
- Syntax-highlighting, tags, tree-based navigation, code-folding
- More correct than regex-based approaches
- *Total!*

```
library
    build-depends base ≥ 4.9
                  ↑ no colon
```

```
(library
  type: (section_type))
(ERROR
  (field_name)
  (identifier)
  (constraint_op)
  (version))
```

Crash slide: Tree-sitter

- Always get a parse tree
- Exposes a DSL for pattern matching

```
library core
    build-depends: base
```

parse tree

```
(cabal
  (sections
    (library
      type: (section_type)
      name: (section_name)
      properties: (property_or_conditional_block
        (field
          name: (field_name)
          value: (field_value
            (identifier)))))))
```

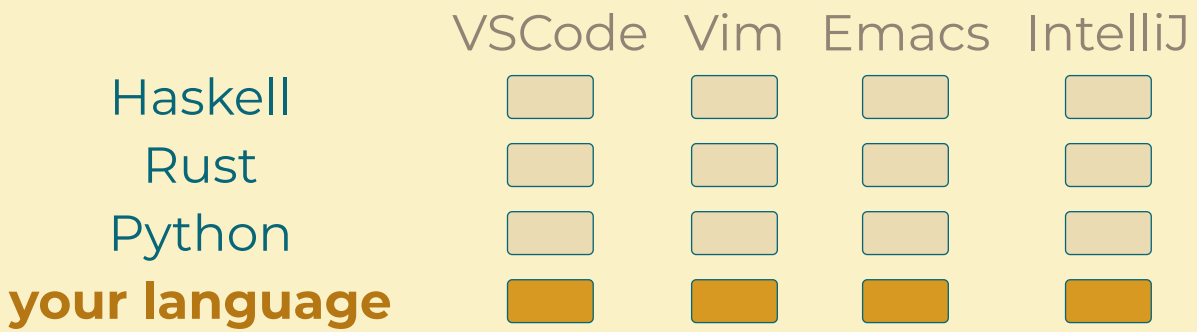
query

```
(library
  name: (section_name) @name)
```

`@name` captures **`core`**

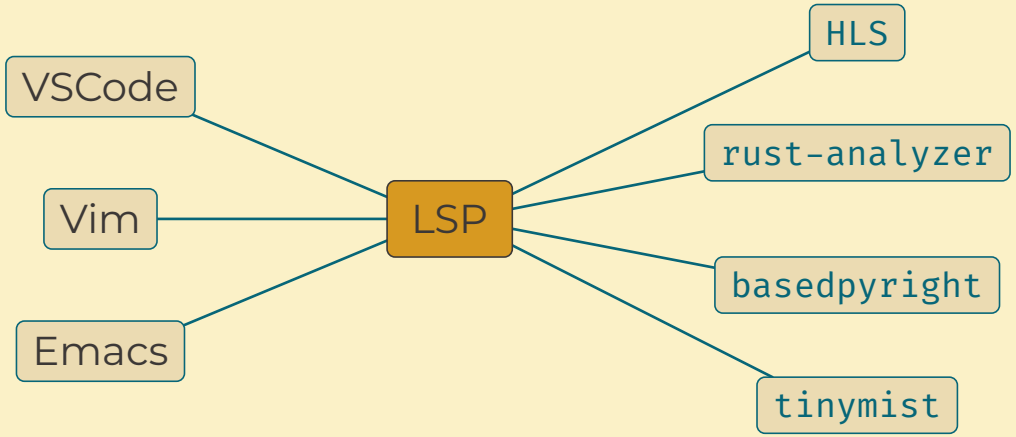
Crash slide: LSP

- How about language-specific features?
- Find references, rename variables, hover, completion-suggestions, ...
- Historically, IDEs like Jetbrains, Eclipse, Visual Studio



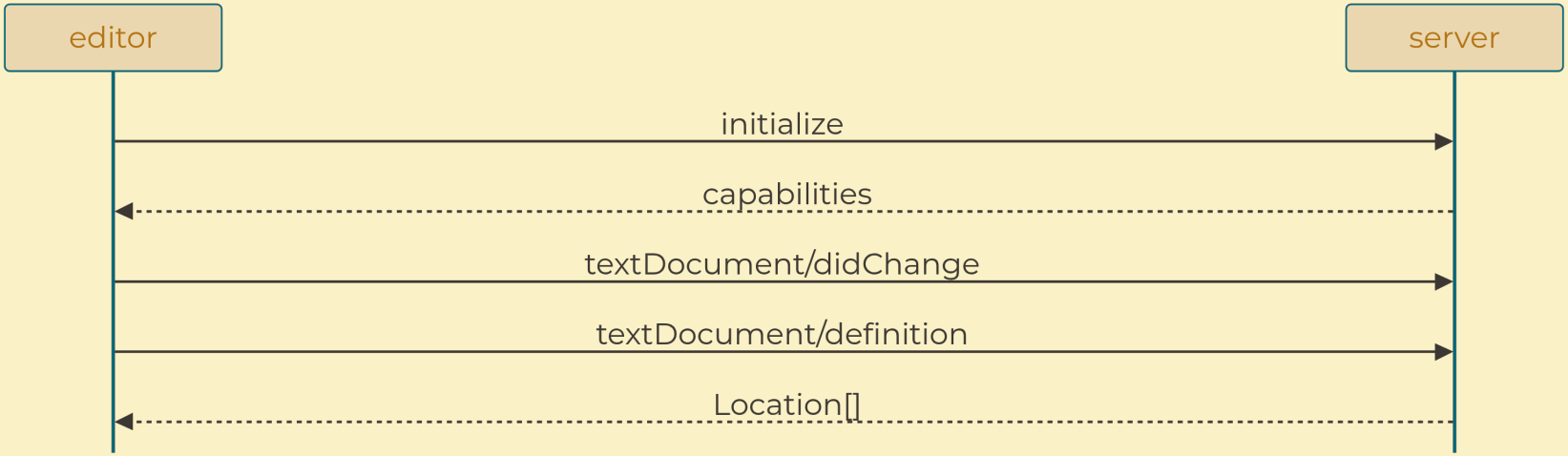
Crash slide: LSP

- Language Server Protocol
- Make it explicit what editors need from implementations
- Editors communicate with language-specific servers
- HLS, rust-analyzer, basedpyright, tinymist



Crash slide: LSP

- Protocol consists of JSON communication
- Communication happens over pipes
- Pre[1]-defined set of capability-based requests/responses



[1] some would say ill-defined, some including me

Connecting the two

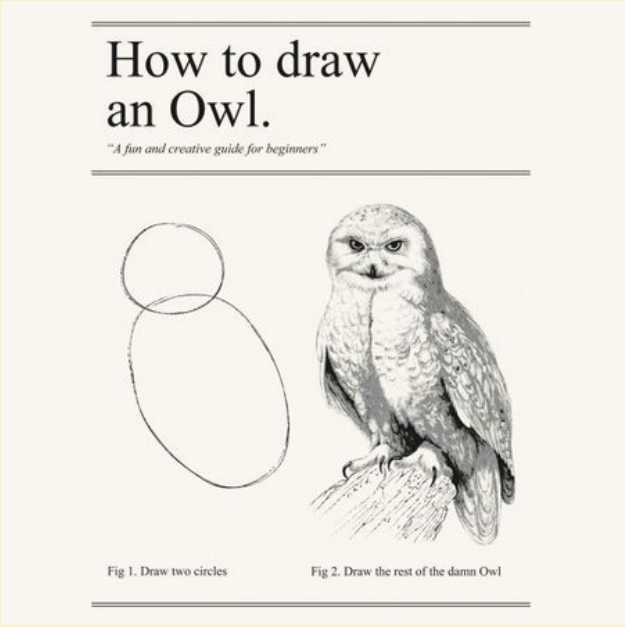
- Tree-sitter gives us parse trees
- Use queries to extract information
- LSP defines the features we need to provide

```
parse :: Text → Tree
query :: Tree → Query → Match
lsp :: Request → Match → Response
```

Connecting the two

- Tree-sitter gives us parse trees
- Use queries to extract information
- LSP defines the features we need to provide

```
parse :: Text → Tree
query :: Tree → Query → Match
lsp :: Request → Match → Response
```



Example

Text

haskell

```haskell
factorial :: Int → Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)
```

python

```python
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Example

Tree

haskell

```haskell
factorial :: Int → Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)
```

python

```python
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Example

Query

source

```
factorial :: Int -> Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)
```

parse tree

```
(declarations
  (signature
    name: (variable)
    type: (function ...))
  (function
    name: (variable)
    patterns: (patterns
      (variable))
    match: (match
      expression: (apply
        function: (variable) ...))
    binds: (local_binds ...)))
```

Example

Query

source

```
factorial :: Int -> Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)

  (signature name: (variable) @name) @definition
  (function name: (variable) @name) @definition
```

parse tree

```
(declarations
  (signature
    name: (variable)
    type: (function ...))
  (function
    name: (variable)
    patterns: (patterns
      (variable))
    match: (match
      expression: (apply
        function: (variable) ...))
    binds: (local_binds ...)))
```

Example

Query

source

```
factorial :: Int -> Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)

  (signature name: (variable) @name) @definition
  (function name: (variable) @name) @definition

  (pattern/variable) @local.definition
  (expression/variable) @local.reference
```

parse tree

```
(declarations
  (signature
    name: (variable)
    type: (function ...))
  (function
    name: (variable)
    patterns: (patterns
      (variable))
    match: (match
      expression: (apply
        function: (variable) ...))
    binds: (local_binds ...)))
```

Example

Match

haskell

```haskell
factorial :: Int → Int
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k = go (acc * k) (k - 1)
```

python

```python
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

```
@name  @local.definition  @local.reference
```

Indexing

- Use queries to capture symbols
- Turn those captures into an index

```haskell
data Capture = Capture
  { text :: Symbol , location :: Span }

captures :: FilePath → Query → [Capture]

type Index = Symbol → Maybe Span
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3   go acc 0 = acc
4   go acc k = ...
```

captures

```
factorial 0:0-0:9
factorial 1:0-1:9
```

index

```
factorial ⟼ 1:0-1:9
```

Goto definition

- Look up the symbol at the cursor

```
goto ::
  Session → Position → Maybe Span
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc 0 = acc
4    go acc k = ...
5 result = factorial 5
```

goto

cursor	symbol	index
5:9	factorial	1:0-1:9

Goto definition

- Look up the symbol at the cursor

```
goto ::
  Session → Position → Maybe Span
```

- The index only contains top-level names
- So we still need something for `go`
- Position relative? Shadowing?

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc 0 = acc
4    go acc k = ...
5 result = factorial 5
```

goto

cursor	symbol	index
5:9	factorial	1:0-1:9
1:14	go	miss

Scoping

- Scopes as subtrees

```haskell
data Scope = Scope
  { binds :: [(Symbol, Span)]
  , parent :: Maybe Scope }

nearestBinder ::
  Ref → Scope → Maybe Span
nearestBinder ref scope =
  lookup ref.text scope.binds <|>
    (nearestBinder ref =<< scope.parent)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3   go acc k = go (acc * k) (k - 1)
```

scopes

	scope	lines	binds
☐	go	3	acc k
☐	factorial	1-3	n go

walk

scope	found

Scoping

- Scopes as subtrees

```haskell
data Scope = Scope
  { binds :: [(Symbol, Span)]
  , parent :: Maybe Scope }

nearestBinder ::
  Ref → Scope → Maybe Span
nearestBinder ref scope =
  lookup ref.text scope.binds <|>
    (nearestBinder ref =≪ scope.parent)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2   where
3     go acc k = go (acc * k) (k - 1)
```

scopes

	scope	lines	binds
	go	3	acc k
	factorial	1-3	n go

walk

	scope	found
go 3:13	go	miss

Scoping

- Scopes as subtrees

```haskell
data Scope = Scope
  { binds :: [(Symbol, Span)]
  , parent :: Maybe Scope }

nearestBinder ::
  Ref → Scope → Maybe Span
nearestBinder ref scope =
  lookup ref.text scope.binds <|>
    (nearestBinder ref =<< scope.parent)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3   go acc k = go (acc * k) (k - 1)
```

scopes

	scope	lines	binds
	go	3	acc k
	factorial	1-3	n go

walk

		scope	found
go	3:13	go	miss
		factorial	3:2-3:4

Scoping

- How do we get those scopes?
- Functions open scopes
- Patterns capture symbols

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2   where
3     go acc k = go (acc * k) (k - 1)
```

locals.scm

```
; a scope per function
(function) @local.scope

; "name" each scope with its binder
(function
  name: (variable) @local.definition
  ; bind in the enclosing scope
  (#set! definition.scope "parent"))

; patterns become definitions
(patterns (variable) @local.definition)

; variables in expressions are references
(expression/variable) @local.reference
```

Goto definition

- Nearest binder first
- Index for the rest

```
resolve ::
  Session → Position → (Scope, Ref)

goto session pos =
  nearestBinder ref scope
    <|> session.index ref.text
  where
    (scope, ref) =
      resolve session pos
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc 0 = acc
4    go acc k = ...
5 result = factorial 5
```

goto

cursor	symbol	found	via
5:9	factorial	1:0-1:9	index
1:14	go	3:2-3:4	scopes

Find references

- References sit inside their binder scope
- Walk down the scope using the tree

```
references name scope =
  filter (named name) scope.refs
  <> foldMap (references name)
       (unshadowed name scope)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc k = go (acc * k) (k - 1)
```

walk

scope	binds	found

Find references

- References sit inside their binder scope
- Walk down the scope using the tree

```
references name scope =
  filter (named name) scope.refs
  <> foldMap (references name)
       (unshadowed name scope)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc k = go (acc * k) (k - 1)
```

walk

	scope	binds	found
☐	factorial	n go	1:14

Find references

- References sit inside their binder scope
- Walk down the scope using the tree

```
references name scope =
  filter (named name) scope.refs
  <> foldMap (references name)
       (unshadowed name scope)
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc k = go (acc * k) (k - 1)
```

walk

	scope	binds	found
☐	factorial	n go	1:14
☐	go	acc k	3:13

Hover

- Use a query to pick what to show
- Can be signature, or binding

signature

```
(signature
  name: (variable) @hover.name) @hover.content
```

binding

```
(function
  name: (variable) @hover.name) @hover.content
```

source

```
0 factorial :: Int → Int
1 factorial n = go 1 n
2  where
3    go acc 0 = acc
4    go acc k =  ...
5 result = factorial 5
```

hover

```
factorial :: Int → Int
```

```
factorial n = go 1 n
 where
  go acc 0 = acc
  go acc k =  ...
```

Selection and folding

- Tree-traversals
- Selection walks up
- Nodes already capture regions

```
selection node =
  node.range :
    foldMap selection node.parent

folds tree =
  [ n.range | n ← nodes tree ]
```

selection

```
go acc k = go (acc * k) (k - 1)
go acc k = go (acc * k) (k - 1)
go acc k = go (acc * k) (k - 1)
go acc k = go (acc * k) (k - 1)
go acc k = go (acc * k) (k - 1)
```

folded

```
0 factorial :: Int → Int
1 factorial n = go 1 n ...
```

Demo

Why is this a bad idea?

- Inherently limited
- No diagnostics beyond lexical
- Cumbersome generic parse trees
- I’m still capturing and assuming semantics
- What even are imports and packages?

Who needs language servers?

- Is it possible to write generic language servers?
- Yes, with limited usefulness
- Best for DSLs, configs, niche languages

Who needs language servers?

- Is it possible to write generic language servers?
- Yes, with limited usefulness
- Best for DSLs, configs, niche languages

Questions?